

Learning Time-Varying Graph Signals via Koopman

Sivaram Krishnan¹, Jinho Choi¹, *Fellow, IEEE*, and Jihong Park², *Senior Member, IEEE*

Abstract—A wide variety of real-world data, such as sea measurements, e.g., temperatures collected by distributed sensors and multiple unmanned aerial vehicles (UAV) trajectories, can be naturally represented as graphs, often exhibiting non-Euclidean structures. These graph representations may evolve over time, forming time-varying graphs. Effectively modeling and analyzing such dynamic graph data is critical for tasks like predicting graph evolution and reconstructing missing graph data. In this paper, we propose a framework based on the Koopman autoencoder (KAE) to handle time-varying graph data. Specifically, we assume the existence of a hidden non-linear dynamical system, where the state vector corresponds to the graph embedding of the time-varying graph signals. To capture the evolving graph structures, the graph data is first converted into a vector time series through graph embedding, representing the structural information in a finite-dimensional latent space. In this latent space, the KAE is applied to learn the underlying non-linear dynamics governing the temporal evolution of graph features, enabling both prediction and reconstruction tasks.

Index Terms—Graph embedding, koopman autoencoder, nonlinear dynamics, prediction and reconstruction, time series analysis, time-varying graphs.

I. INTRODUCTION

A. Motivation

GRAPHS are fundamental data structures for modeling the structure and interactions within complex systems [1] across a variety of domains, including, but not limited to, social networks [2], biological systems [3], transportation networks [4], and communication systems [5]. These data structures provide a versatile framework for representing relationships and dependencies, enabling insights into the organization and behavior of complex systems. In many real-world applications, the underlying graph data is not static; instead they evolve over time. *Time-varying graphs* [6] are a type of graph data characterized by temporal variations in their components or overall configuration. Unlike the commonly studied static graph structures, analyzing time-varying graph data introduces additional challenges. In this work, we consider two application scenarios involving time-varying graphs, as follows:

- the reconstruction of graph signals
- the prediction of graph signals.

While the reconstruction of graph signals is necessary for recovering missing information, which is common in real-world sensor networks or data transmission scenarios, prediction, on the other hand, enables forecasting the future states of the systems and thus supports planning, decision-making, and control in dynamical environments.

While several methods exist in the literature for graph signal reconstruction, which is based on the graph signal processing (GSP) [7], [8] or graph neural network (GNN) paradigm [9], most of the formulation is limited to time-varying graphs with a fixed structure. Another challenge for graph signal prediction is that most of the data-driven methods are computationally heavy and often fail to capture the non-linear dynamics inherent in time-varying graphs.

To address these limitations, we propose a novel framework leveraging the *Koopman operator theory* [10]. This theory demonstrates that non-linear dynamics can be represented linearly by focusing on functions (observables) of the system's state, enabling linear modeling in a higher-dimensional latent space. The Koopman autoencoder (KAE) [10], [11], [12] offers a data-driven approach to linearize complex non-linear dynamical systems by learning such linear representations in a lifted (latent) space. This physics-informed approach ensures predictions adhere to a consistent and interpretable dynamic structure, rather than merely fitting data in a black-box manner. Finite-dimensional approximations [13] and Koopman invariant subspaces [14] have further advanced this field, with comprehensive reviews available in [15], [16] and deep learning applications explored in [12], [17].

Building upon our previous work on Koopman-theoretic graph embeddings and the graph KAE (GKAE) [18], [19]¹, we introduce a robust framework for predicting time-varying graphs and reconstructing missing graph data. Our approach assumes the observed graph sequence arises from an underlying, unknown non-linear dynamical system. For learning on these graphs, the data is first embedded into a latent space (graph embedding) as a multivariate time series, capturing structural information. The GKAE then learns the linear dynamics governing the evolution of these graph embeddings, enabling both tasks.

The contributions of this paper are summarized as follows:

- C1) We formally categorize time-varying graphs into two broad types: those with time-varying graph signals and

¹ This paper is an extended version of [19], providing a generalized framework for prediction and reconstruction with comprehensive exposition and extensive experimental validation.

Received 12 January 2025; revised 16 July 2025 and 1 September 2025; accepted 1 December 2025. Date of publication 2 January 2026; date of current version 6 January 2026. The associate editor coordinating the review of this article and approving it for publication was Dr. Wei Hu. (*corresponding author: Sivaram Krishnan.*)

Sivaram Krishnan and Jinho Choi are with the School of Electrical and Mechanical Engineering, The University of Adelaide, Adelaide, SA 5005, Australia (e-mail: sivaram.krishnan@adelaide.edu.au; jinho.choi@adelaide.edu.au).

Jihong Park is with the Information Systems Technology and Design Pillar, Singapore University of Technology and Design, Singapore 487372 (e-mail: jihong_park@sutd.edu.sg).

Digital Object Identifier 10.1109/TSIPN.2025.3642225

those with time-varying graph structures. We highlight the limitations of existing GSP-based and GNN-based approaches for reconstructing time-varying graph signals, which is majorly limited to only time-varying graph signals with a static graph structure.

- C2) We propose the GKAE as a robust framework for reconstruction and prediction in time-varying graphs, capable of handling diverse types of time-varying graphs. The GKAE leverages GNN and Koopman theory, operating in the latent space to model non-linear dynamics through a linear Koopman matrix, making it applicable to all types of time-varying graphs.
- C3) We evaluate our method against established baselines. For reconstruction tasks, we compare it with GSP among other approaches, while for prediction tasks, we benchmark it against other data-driven Koopman-based methods and GNN-based state-of-the-art methods. Our results demonstrate superior performance in terms of both reconstruction error and prediction error using GKAE across various types of time-varying graphs.

The comprehensive glossary of notations used throughout the paper is summarized in Table I.

B. Related Work

We begin by reviewing methods developed for graph signal reconstruction, with a particular emphasis on approaches grounded in GSP. We then examine prediction methods—often used interchangeably with forecasting in the literature—which are majorly data-driven approaches, particularly deep learning and GNN-based models

a) *Reconstruction of Graph Signals:* GSP extends traditional signal processing concepts to graph-based data [20], [21]. Unlike traditional signal processing, which operates on regular grids with established frameworks like Fourier transforms and sampling theories (e.g., the Nyquist rate), GSP addresses signals defined on non-uniform and often dynamic graph structures that represent underlying networks [22], [23]. GSP relies on spectral analysis through operations such as the graph Fourier transform, focusing on processing graph signals in their native domain. However, GSP faces computational challenges, particularly when dealing with large or time-varying graphs.

GSP-based methods have traditionally been developed with a focus on the graph signal reconstruction problem. For instance, the problem of time-varying graph signal reconstruction is addressed in [7], [8]. In [7], the definition of smooth signals was extended from static to time-varying graph signals, and an optimization problem was formulated for their reconstruction. In contrast, [8] generalized this formulation by incorporating Sobolev smoothness. The formulation of these methods are limited to time-varying graphs with a static structure. To address this, a notable robust method was proposed in [24] for recovering time-varying graph signals from corrupted data in dynamic physical sensor networks, where it is assumed that the time-varying graph structure is known a priori. The core contribution is a constrained convex optimization framework designed to handle missing values and outliers. This is achieved

TABLE I
GLOSSARY OF NOTATIONS

Symbol	Description
Graph Notations	
$G(t)$	Graph at time step t .
$\mathcal{V}$	Set of nodes.
$\mathbf{x}(t)$	Graph signals at time step t .
$\mathcal{E}(t)$	Edge set at time step t .
$\mathbf{A}(t), \mathbf{W}(t)$	Binary/Weighted Adjacency matrix at time t .
$\mathcal{N}_l(t)$	Neighborhood of node l at time t .
$\mathbf{L}(t) = \mathbf{D}(t) - \mathbf{A}(t)$	Laplacian matrix
Model Notations	
$\mathbf{g}(t)$	Graph embedding/state vector at time t .
$\mathbf{K}$	Koopman matrix of dimension M .
$\mathbf{h}(t)$	Koopman variable at time t .
L	Maximum prediction step during training.
$\mathcal{E}, \mathcal{D}$	Encoder and decoder function.
$\theta_{\text{GNN}}, \theta_{\text{KAE}}, \theta_{\text{LC}}$	Parameters for encoder functions.
$\phi_{\text{GNN}}, \phi_{\text{KAE}}, \phi_{\text{LC}}$	Parameters for decoder functions.
$\text{POOL}(\cdot)$	Pooling function.
$\mathbf{Y}$	Sampled graph signals.
$\mathbf{J}$	Sampling matrix.
$\mathbf{y}'(t)$	Sampled graph signals at time t .
$\mathbf{W}'(t)$	Sampled adjacency matrix at time t .
Loss Functions and Performance Metrics	
$\mathcal{L}_{\text{GKAE}}$	Loss function for the graph Koopman autoencoder.
$\mathcal{L}_{\text{LC}}$	Loss function for the Latent consistency autoencoder.
ϵ_{pred}	Average prediction error.
ϵ_{reco}	Average reconstruction error.
P	Total number of prediction steps after training.
β_1, β_2	Loss hyperparameters.
Dataset	
N	Total number of nodes.
k	Number of neighbors using k -nearest neighbors.
r	Communication radius for determining the neighborhood.

by incorporating both graph Laplacian regularization, which enforces signal smoothness across the known dynamic graph, and temporal regularization, which promotes smoothness over time. The problem is then solved efficiently using a primal-dual splitting method. However, this method relies on the assumption of signal smoothness, similar to other approaches in [7], [8], which may not be valid for data obtained from complex real-world systems. Additionally, the temporal regularization techniques typically require complete knowledge of the signal, and the associated graph Laplacian is assumed to be known over time. A broader challenge lies in analyzing time-varying graph data directly in the graph domain, as this involves dealing with the high dimensionality of non-Euclidean spaces.

On the other hand, *graph embedding* methods [25], [26] offers a fundamentally different approach. Graph embedding methods first map the non-Euclidean graph data into a lower-dimensional latent space while preserving its structural properties. While GSP relies heavily on spectral analysis of the graph and operates directly in the node or edge domain, graph embedding methods aim to learn compact vector representations of nodes, edges, or the entire graph. These representations capture both local and

global graph information, allowing for more efficient processing of complex, large-scale, or time-varying graphs. Several graph embedding methods exist that vary over how they handle the input, the type of output, and how the information propagates through the graph. Spectral embedding method [27] leverages the eigenvalues and the eigenvectors of graph-related matrices for determining a lower-dimensional Euclidean space, capturing the global structure of the graph and local neighborhoods of the nodes. Other methods [28], [29], [30] use random walks for capturing the neighborhood of the nodes in a graph. While matrix factorization methods [31] focus on preserving the high-order proximity by factorizing a general similarity matrix. With the success of neural networks, GNN-based methods are the state-of-the-art for processing graph-structured data [32]. Prominent examples include the seminal works on graph convolutional networks (GCNs) [33] and graph attention networks (GATs) [34], which leverage convolution and attention mechanisms, respectively, to learn powerful representations of graph data. Although GCN and GAT are effective for learning feature representations on graphs, they often perform poorly when applied to graph signal reconstruction tasks as a standalone architecture. In [35], graph unrolling networks are employed for graph signal reconstruction. A known characteristic of such iterative methods is that they often require a high number of parameters to achieve convergence. While in [9], [36], the authors propose the Gegenbauer-GNN (GegenGNN), which is an encoder-decoder architecture using cascaded Gegenbauer polynomials filters in its convolution layer. The loss function for training includes the Sobolev smoothness regularization, as in [8]. It is important to note that these aforementioned GNN-based methods were originally formulated for time-varying graphs with a fixed structure.

Beyond reconstruction, the effective forecasting of time-varying graph signals presents a non-trivial task, which we now review.

b) Forecasting of Graph Signals: As our method builds upon Koopman theory, we firstly review existing Koopman-based techniques. Although several Koopman-based methods exist [12], [37], [38], none have been adapted to handle time-varying graphs. In such instances, we have to predict sequentially for each node, which significantly increases the computational complexity or neglect the spatial interactions all together, which leads to performance loss. On the other hand, recurrent neural networks (RNNs) are prominent state-of-the-art architectures in learning on temporal data but they are often computationally heavy and fail to capture long-term dependencies [39]. Other approaches such as long-short term memory (LSTM) [40] and gated-recurrent units (GRU) [41] have been proposed for ameliorating the aforementioned challenges, inherent in RNN-based architectures.

Other prominent methods for graph signal forecasting relies on data-driven GNN-based approaches. For instance, spatio-temporal GCN (STGCN) [42] introduces a unified framework that combines spatial graph convolutions with temporal convolutional layers to model both spatial dependencies among nodes and temporal patterns in sequential data. Diffusion convolution RNN (DCRNN) [43] embeds diffusion convolutions within a GRU cell. This enables the model to capture spatio-temporal

dependencies by treating the process as a diffusion on the graph that evolves through the recurrent state updates. Other sequence-to-sequence methods employ an encoder-decoder framework that leverages recurrent models to capture temporal dependencies within graph-embedded data. Continuous Product GCN (CITRUS) [44] performs joint multidomain learning by defining convolutional operations on a Cartesian product graph. This product graph is formed by combining the spatial graph—representing connections among spatial nodes—with a temporal graph that encodes relationships across time steps. The resulting unified graph's nodes correspond to specific spatial locations at particular time instances. CITRUS applies learnable continuous heat kernels on this product graph, which act as filters modeling how information diffuses smoothly across both spatial and temporal dimensions.

c) Main Differences: Unlike conventional overparameterized models, we propose an efficient approach tailored for natural systems. By enforcing a linear latent space, the model learns the system's underlying dynamics—approximating the Koopman operator—rather than fitting input-output mappings. This yields interpretable predictions grounded in physical structure, analyzable via the spectral decomposition of the learned Koopman matrix.

The structure of the paper is as follows: Section II introduces our framework for time-varying graphs and their associated graph signals, categorizing them based on the components of the graph that change over time. It also outlines the limitations of existing approaches. Section III presents our proposed method for both prediction and reconstruction tasks on time-varying graphs. The problem formulations for these tasks are provided in Sections III-C and IV, respectively. Section V describes the experimental setup, reports the results of our approach, and compares its performance against established baselines. Finally, Section VI concludes the paper.

II. BACKGROUND: TIME-VARYING GRAPHS

In this section, we briefly present an overview of time-varying graphs and associated graph signals.

A. Properties of Graphs

A graph is represented as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, 2, \dots, N\}$ is the set of the nodes (or vertices) and $\mathcal{E} = \{(l, m) : l, m \in \mathcal{V}\}$ denotes the set of the edges [45]. Here, N is the number of nodes. The set of neighbors of node l is denoted by $\mathcal{N}_l = \{m \mid (l, m) \in \mathcal{E}\}$, and for unweighted graphs, the degree of node l is given by $d_l = |\mathcal{N}_l|$, i.e., the number of neighbors.

For a given graph, the $N \times N$ Laplacian matrix is defined as

$$\mathbf{L} = \mathbf{D} - \mathbf{A}, \quad (1)$$

where $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_N)$ and $\mathbf{A}$ is the $N \times N$ adjacency matrix, which is given by

$$[\mathbf{A}]_{l,m} = \begin{cases} 1, & \text{if } (l, m) \text{ or } (m, l) \in \mathcal{E}; \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

For an undirected graph, $\mathbf{A}$ is symmetric, while $\mathbf{A}$ is asymmetric for a directed graph. If each node has the same degree, i.e., $d_1 = \dots = d_N = d$, the graph is called a d -regular graph.

A weighted graph is a graph with edge weights. For an edge in $\mathcal{E}$, a non-negative weight can be given, which is denoted by $w_{l,m}$. Thus, the weight matrix, denoted by $\mathbf{W}$, where $[\mathbf{W}]_{l,n} = w_{l,n}$, can be seen as a weighted adjacency matrix, and a weighted graph is represented as $G = (\mathcal{V}, \mathcal{E}, \mathbf{W})$. The degree of a node in a weighted graph is now defined as the sum of the weights of all the edges connected to it. Consequently, the Laplacian matrix of a weighted graph becomes

$$\mathbf{L} = \mathbf{D} - \mathbf{W}. \quad (3)$$

If $w_{l,m} = 1$ for $(l, m) \in \mathcal{E}$, then $\mathbf{W} = \mathbf{A}$. Thus, the weighted graph is a generalized version of the conventional graph. For undirected graphs, $\mathbf{L}$ is symmetric. In addition, the eigenvalues of $\mathbf{L}$ are nonnegative, meaning that $\mathbf{L}$ is a positive semidefinite matrix [45]. Thus, the eigendecomposition of $\mathbf{L}$ becomes

$$\mathbf{L} = \mathbf{U}\mathbf{\Sigma}\mathbf{U}^T, \quad (4)$$

where $\mathbf{\Sigma}$ is the diagonal matrix of the eigenvalues, $\{\lambda_l \geq 0\}$, i.e., $\mathbf{\Sigma} = \text{diag}(\lambda_1, \dots, \lambda_N)$, and $\mathbf{U}$ is the matrix whose column vectors are the eigenvectors, i.e., $\mathbf{U} = [\mathbf{u}_1 \dots \mathbf{u}_N]$. Here, $\mathbf{u}_l$ stands for the eigenvector corresponding to λ_l .

Let λ_n denote the n th smallest eigenvalue. For a connected graph, we have $\lambda_1 = 0$ and $\lambda_2 > 0$ [20], [45], i.e.,

$$0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_N.$$

Let the volume of the graph be $\text{vol}(G) = \text{tr}(\mathbf{L}) = \sum_{n=1}^N \lambda_n$. Then, it can be shown that

$$\lambda_2 \leq \frac{\text{vol}(G)}{N-1}, \quad (5)$$

where the equality can be achieved when $\lambda_2 = \dots = \lambda_N$.

B. Graph Signals

In the context of GSP, a graph signal signifies scalar values over the nodes of a graph such that, $\mathbf{x} = [x_1 \dots x_N]^T \in \mathbb{R}^N$. In [46], the graph Fourier transform of $\mathbf{x}$ is defined as

$$\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}, \quad (6)$$

and since $\mathbf{U}$ is unitary (for undirected graphs), the inverse graph Fourier transform is given by

$$\mathbf{x} = \mathbf{U} \hat{\mathbf{x}}. \quad (7)$$

In [20], the following graph Laplacian quadratic form [47] has been considered for the global smoothness:

$$\begin{aligned} S_2(\mathbf{x}) &= \mathbf{x}^T \mathbf{L} \mathbf{x} \\ &= \sum_{(l,n) \in \mathcal{E}} w_{l,n} (x_l - x_n)^2. \end{aligned} \quad (8)$$

It can be shown that

$$\begin{aligned} S_2(\mathbf{x}) &= \mathbf{x}^T \mathbf{U} \mathbf{\Sigma} \mathbf{U}^T \mathbf{x} \\ &= \hat{\mathbf{x}}^T \mathbf{\Sigma} \hat{\mathbf{x}} = \sum_{n=1}^N \lambda_n |\hat{x}_n|^2, \end{aligned} \quad (9)$$

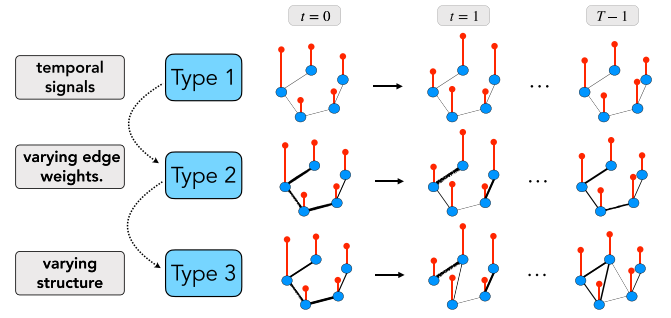


Fig. 1. Types of time-varying graphs: Type-1 varies in signals only, Type-2 in signals and edge weights, and Type-3 in signals, edge weights, and structure.

For a connected graph, since $\lambda_1 = 0$, the graph Laplacian quadratic form is bounded as follows:

$$0 \leq S_2(\mathbf{x}) \leq \lambda_N \|\mathbf{x}\|^2. \quad (10)$$

Moreover, if $\mathbf{x}$ is orthogonal to the constant vector (i.e., $\sum_{i=1}^N x_i = 0$), then the tighter bound holds:

$$\lambda_2 \|\mathbf{x}\|^2 \leq S_2(\mathbf{x}) \leq \lambda_N \|\mathbf{x}\|^2. \quad (11)$$

C. Time-Varying Graphs

There are different types of time-varying signals associated with given graphs. Assuming that the number of nodes is fixed (or $\mathcal{V}$ is fixed), we categorize time-varying graphs into the following three types:

- *Type-1*: The signals $\mathbf{x}$ can be extended over time as time-varying graph signals or as a time series, which are referred to as time-varying graph signals [7]. Throughout the paper, the time-varying graph signal is referred to as type-1. It is noteworthy that the graph structure is time-invariant, i.e., $\mathcal{E}$ and $\mathbf{W}$ remain invariant over time, while the node features, i.e., $\mathbf{x}(t)$, are time-varying.
- *Type-2*: While the graph structure is invariant over time for type-1 graphs, the edge weights can vary. In type-2 graphs, we assume that $\mathcal{V}$ and $\mathcal{E}$ are invariant, while the weights are time-varying, i.e., $\mathbf{W}(t)$.
- *Type-3*: The connectivity of a graph can vary over time, i.e., $\mathcal{E}$ becomes $\mathcal{E}(t)$. The resulting graph is referred to as a type-3 time-varying graph, and the time-varying signal becomes a three-tuple $(\mathbf{x}(t), \mathcal{E}(t), \mathbf{W}(t))$. According to [48], type-1 graphs are referred to as attributed graphs, while type-3 graphs are referred to as spatial-temporal graphs.

In Fig. 1, we illustrate the relationship between different types of time-varying graphs, where type-1 graphs are a subset of type-2 graphs, and type-2 graphs are a subset of type-3 graphs. Note that if a graph is fully connected, types 2 and 3 are equivalent.

D. Existing Problem Formulation for Graph Signal Reconstruction

For a finite-length sequence of a graph signal $\mathbf{x}(t)$, let $\mathbf{X} = [\mathbf{x}(0) \dots \mathbf{x}(T-1)] \in \mathbb{R}^{N \times T}$ represent the complete signal. We assume that the data is fully available for the first τ time steps, after which only a subset of the graph signal is available at each

time t . Let $\mathcal{A}(t)$ denote the index set of nodes with measurements available at time t . The available graph signals are then expressed as

$$\mathbf{Y} = \mathbf{J} \odot \mathbf{X} \in \mathbb{R}^{N \times T}, \quad (12)$$

where $\odot$ represents the Hadamard product and $\mathbf{J} \in \mathbb{R}^{N \times T}$ is the sampling matrix, given as

$$[\mathbf{J}]_{n,t} = \begin{cases} 1, & \text{if } t < \tau \\ 1, & \text{if } n \in \mathcal{A}(t) \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

Note that $\mathcal{A}(t)$ may vary with time t .

Recent GSP-based methods [7], [8], [24], along with GNN-based approaches such as [9], extend the notion of signal smoothness to the temporal domain [23]. For instance, spatial smoothness can be characterized using the Laplacian matrix, as illustrated in (8). For time-varying graph signals, temporal smoothness can also be defined in terms of the Laplacian matrix. To explain this approach, let $\tilde{\mathbf{x}}(t) = \mathbf{x}(t) - \mathbf{x}(t-1)$ for $t \in \{1, \dots, T-1\}$, which denotes the temporal difference signal. From (8), a set of temporally smooth graph signals, which are called the ϵ -structured time-varying graph signals, can be defined as

$$\Omega_\epsilon = \left\{ \mathbf{X} : \sum_{t=1}^{T-1} S_2(\tilde{\mathbf{x}}(t)) \leq (T-1)\epsilon \right\}. \quad (14)$$

The following optimization is widely considered for time-varying graph reconstruction:

$$\min_{\mathbf{X}} S_2(\tilde{\mathbf{x}}(t)) + \|\mathbf{J} \odot \mathbf{X} - \mathbf{Y}\|_{\text{F}}^2, \quad (15)$$

where $\|\cdot\|_{\text{F}}$ is the Frobenius norm.

As detailed in Section I-B, the methods in [7], [8], [9], [36] are limited to type-1 graphs. Even approaches like [24], which target time-varying graphs, assume full knowledge of the dynamic structure and rely on simplistic temporal differences—making them insufficient for capturing complex, non-linear dynamics. Additionally, as the number of time steps T increases, the term $(T-1)\epsilon$ grows proportionally, permitting larger deviations from smoothness across the entire time horizon and thus adding complexity to the reconstruction process.

In the following section, we formally introduce our approach, starting with the prediction task. As reconstruction builds upon the predicted signals, we present the reconstruction framework thereafter.

III. GKAE FOR TIME-VARYING GRAPH SIGNALS: ARCHITECTURE AND TRAINING

In this section, after introducing the Koopman operator and its autoencoder implementation, we present our proposed approach, GKAE. Our model effectively handles non-Euclidean, time-varying graph data (including Type-3 signals) and offers an interpretable approach for predicting and learning their inherent dynamics.

A. Koopman Operator Theory

The Koopman operator theory provides a powerful framework for linearizing the evolution of a non-linear dynamical system [10], [15], [16], [49]. In this subsection, we present an overview of the Koopman operator theory.

Denote by $\mathbf{g}(t)$ the state of a non-linear dynamical system that evolves over time as follows:

$$\mathbf{g}(t+1) = \mathbf{F}(\mathbf{g}(t)) \in \mathcal{G} \subseteq \mathbb{R}^N, \quad (16)$$

where $\mathbf{F}$ represents the flow map, a non-linear and unknown function that governs the dynamics of the system and $\mathcal{G}$ is the state space, which is a subset of the N -dimensional vector space. According to the Koopman operator theory, given a measurement function, $h : \mathcal{G} \rightarrow \mathbb{R}$, which is an observable function, there exists a linear operator called the *Koopman operator*, denoted by $\mathcal{K}$, which can be applied to all such observable h , to advance them in time, as follows:

$$\mathcal{K}h = h \circ \mathbf{F}, \quad (17)$$

where $\circ$ represents the composition operator, existing on a smooth manifold. Applying (17) to (16), we have

$$h(\mathbf{g}(t+1)) = h \circ \mathbf{F}(\mathbf{g}(t)) = \mathcal{K}h(\mathbf{g}(t)), \quad (18)$$

where $h(\mathbf{g}(t))$ is an observable measured at time t . This can be extended to the case with multiple observables. Precisely, let $\mathbf{h}(t) = [h_1(t) \dots h_M(t)]^T$, where $h_m(t) = h_m(\mathbf{g}(t))$. Then, from (18), we have

$$\mathbf{h}(t+1) = \mathcal{K}\mathbf{h}(t). \quad (19)$$

We can readily show that the Koopman operator is linear, since $\mathcal{K}(c_1\mathbf{h}_1(t) + c_2\mathbf{h}_2(t)) = c_1\mathbf{h}_1(t+1) + c_2\mathbf{h}_2(t+1)$, where $c_1, c_2 \in \mathbb{R}$.

If $\mathbf{h}(t) \in \mathcal{H}$ and $\mathcal{K}\mathbf{h}(t) \in \mathcal{H}$, where $\mathcal{H}$ is a finite-dimensional space, $\mathcal{H}$ becomes a Koopman invariant subspace [14]. In this case, the Koopman operator in (19) becomes a square matrix, denoted by $\mathbf{K}$.

B. Koopman Autoencoder

Although the Koopman operator is useful for linearizing non-linear dynamical systems, which aids in modeling and prediction, finding the Koopman invariant subspace is challenging. Data-driven techniques, such as dynamic mode decomposition (DMD) methods [50], [51] and deep learning based approaches [12], [17], have been developed to address this challenge. In particular, the approaches in [12], [17] are based on the autoencoder architecture, and called the KAE, which consists of the three main components: an encoder, a Koopman matrix, and a decoder, where the encoder and decoder are neural networks. It is designed to estimate the Koopman operator and its invariant subspace, capturing the smooth dynamics through a linear operator.

Let $\mathcal{E}_{\theta_{\text{KAE}}}(\mathbf{g})$ be the encoder that maps the state vector $\mathbf{g}$ to a latent variable, denoted by $\mathbf{h} \in \mathbb{R}^M$, where M is the dimension of the latent space. Here, θ_{KAE} represents the parameter vector for the encoder of the KAE. Then, we assume that the encoder

performs the linearization so that the following relation can hold:

$$\begin{aligned}\mathcal{E}_{\theta_{\text{KAE}}}(\mathbf{g}(t+1)) &= \mathbf{h}(t+1) \\ &= \mathbf{K}\mathbf{h}(t) = \mathbf{K}\mathcal{E}_{\theta_{\text{KAE}}}(\mathbf{g}(t)),\end{aligned}\quad (20)$$

where $\mathbf{K}$ is the Koopman matrix. By allowing the inverse mapping with the decoder, denoted by $\mathcal{D}_{\phi_{\text{KAE}}}$, we can have the original state vector as follows:

$$\mathbf{g}(t) = \mathcal{D}_{\phi_{\text{KAE}}}(\mathbf{h}(t)), \quad (21)$$

where ϕ_{KAE} is the parameter vector of the decoder. Here, both the encoder and decoder are assumed to be neural networks. From (20) and (21), using the linearity, we can derive the following relation:

$$\mathbf{g}(t+l) = \mathcal{D}_{\phi_{\text{KAE}}}(\mathbf{K}^l \mathcal{E}_{\theta_{\text{KAE}}}(\mathbf{g}(t))), \quad l = 0, 1, \dots \quad (22)$$

This relation holds for all $l \geq 0$ if an encoder can be designed to map the state vector into the Koopman invariant subspace, and the decoder serves as the inverse function of the encoder. According to Koopman operator theory, the dimension of the latent space, M , can be infinite. However, in practice, M is finite, meaning the relation in (22) holds approximately and only for a finite number of distinct values of l .

Consequently, with a given set of state vectors, $\{\mathbf{g}(0), \dots, \mathbf{g}(T-1)\}$, it may be possible to determine the parameter vectors, θ_{KAE} and ϕ_{KAE} , and the Koopman matrix, $\mathbf{K}$, by minimizing the error or loss defined as:

$$\begin{aligned}\mathcal{L}_{\text{KAE}}(\theta, \phi, \mathbf{K}, L) \\ = \sum_{l=0}^{L-1} \sum_t \|\mathbf{g}(t+l) - \mathcal{D}_{\phi_{\text{KAE}}}(\mathbf{K}^l \mathcal{E}_{\theta_{\text{KAE}}}(\mathbf{g}(t)))\|^2,\end{aligned}\quad (23)$$

where L is a finite positive integer representing the maximum prediction step.

If the Koopman encoder successfully maps the state vector into the Koopman invariant subspace, (22) holds for any $l \geq 0$ due to global linearization. However, with the finite dimensionality of the latent space, the encoder can only approximate the Koopman invariant subspace. Consequently, when L is small, the normalized loss, $\frac{1}{L}\mathcal{L}_{\text{KAE}}$, may be small, but the approximation is not as effective. Increasing L ensures a better linearization, as a larger value allows for a more accurate approximation of the Koopman invariant subspace over a longer time horizon.

As mentioned earlier, the dimension of the latent space, M , is ideally infinite according to Koopman operator theory and is typically expected to be larger than N , the dimension of the state space, especially when modeling non-linear dynamical systems where a high-dimensional latent representation is necessary to capture the full dynamics. However, in some cases, M can be smaller than N , resulting in the KAE having an hourglass shape, where the low-dimensional latent space is designed to extract key features or dominant modes of the system, serving as a compact representation of its underlying dynamics [16]. This dimensionality reduction is advantageous for applications such as modeling multivariate time series, where a simplified yet informative model is sufficient.

C. GKAE Application 1 : Prediction on Time-Varying Graphs

The time-varying graph data is represented as a sequence of spatio-temporal graphs

$$\mathbb{G} = [G(0), \dots, G(T-1)], \quad (24)$$

where for each time step t , $G(t) = (\mathbf{x}(t), \mathcal{E}(t), \mathbf{W}(t))$. While the traditional KAE is effective in modeling non-linear dynamics using an autoencoder architecture [12], [17], it is typically applied to data originating from a single node, modeled as a Euclidean sequence of input states. This inherently limits its applicability to non-Euclidean data, such as spatio-temporal graphs, where the dynamics arise from interactions among multiple nodes in a graph structure. To extend the modeling capability of the KAE to non-Euclidean sequences, we propose transforming the non-Euclidean graph data into a Euclidean-compatible input. This is achieved by learning a graph embedding that captures the spatial and temporal relationships between nodes at each time step. The graph embedding serves as a compact state vector for each spatio-temporal graph, enabling the KAE to model the dynamics of the sequence effectively while preserving the underlying graph structure and relationships.

While various techniques exist for converting a graph realization into a graph embedding, we adopt an autoencoder architecture, which has demonstrated significant success [52], [53] in capturing high-dimensional graph structures and representing them as compact graph embeddings. This approach enables the effective transformation of complex graph data into a lower-dimensional latent space while preserving the essential spatial and temporal relationships within the graph. We input the encoder and decoder on either side of the KAE as shown in Fig. 2, which is thereafter referred to as the graph encoder and graph decoder, denoted by $\mathcal{E}_{\theta_{\text{GNN}}}$ and $\mathcal{D}_{\phi_{\text{GNN}}}$, respectively. The GKAE consists of five components: a graph encoder, a Koopman encoder, a Koopman matrix, a Koopman decoder, and a graph decoder.

The operations within the GKAE can be described using the following components:

1) *Graph Encoder*: The graph encoder decomposes each graph realization as a graph embedding, which aims at finding the latent vectors preserving the spatial characteristics of the graphs. The latent variable allows us to represent the dynamical system of multiple nodes as an Euclidean sequence of input, enabling the use of KAE to linearize the dynamics of the latent variables, which serve as state vectors for the time-varying graphs. We use a pooling operator for a compact representation as follows:

$$\mathbf{g}(t) = \text{POOL}(\mathcal{E}_{\theta_{\text{GNN}}}(\mathbf{x}(t), \mathcal{E}(t), \mathbf{W}(t))), \quad (25)$$

where $\text{POOL}(\cdot)$ represents an average pooling function.

2) *KAE*: The KAE uses the input graph embedding, which is presumably non-linear due to the non-linearity inherent in the original dynamics and the graph encoder. With processable input of the graph embedding as the state vector of the dynamical system, we linearize using (20) and the future embedding predictions are found using the Koopman matrix, as seen in (22) and (21).

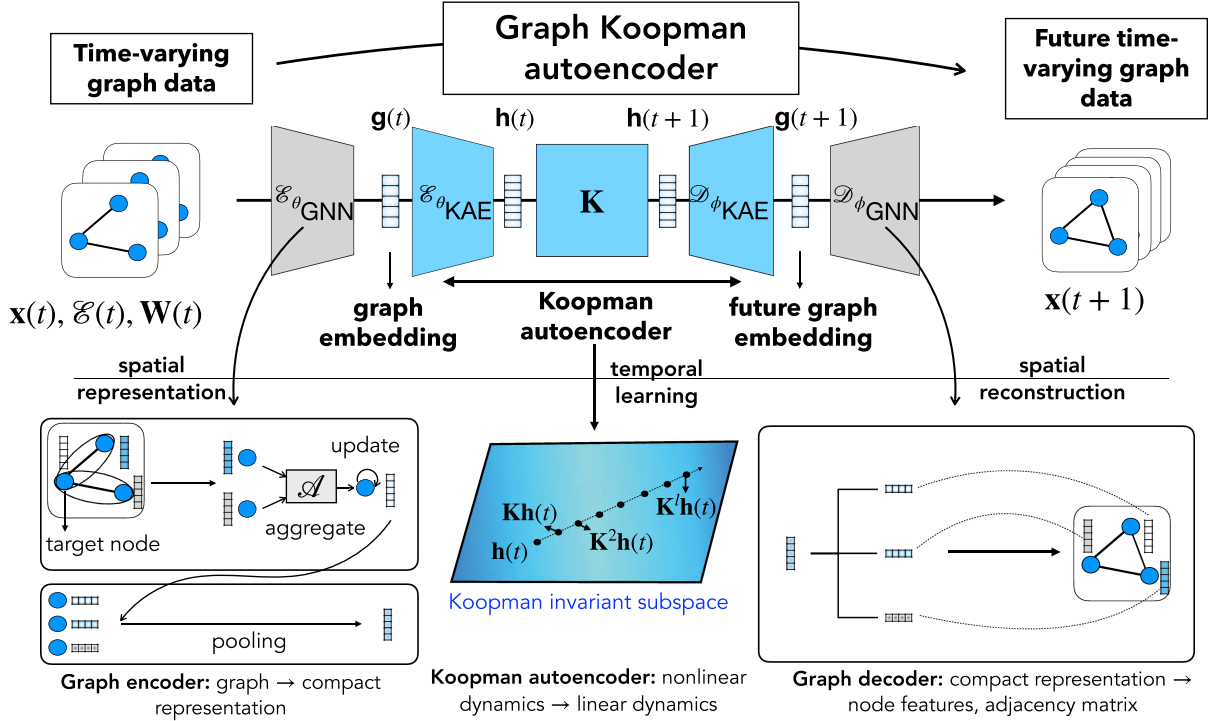


Fig. 2. Schematic of the proposed Graph Koopman Autoencoder (GKAE) framework. The graph encoder aggregates ($\mathcal{A}$) and updates graph signals based on neighbors to generate a graph embedding. The Koopman autoencoder predicts future embeddings linearly, and the graph decoder reconstructs the graph in its original space.

3) *Graph Decoder:* A graph decoder aims at reconstructing from the graph embedding state to the original graph state. Using the input as the graph embedding at any time step, we can reconstruct the node features as

$$\hat{\mathbf{x}}(t+1) = \mathcal{D}_{\phi_{\text{GNN}}}(\mathbf{g}(t+1)), \quad (26)$$

where using the reconstructed node features, the adjacency matrix can be derived based on a predefined rule for edge formation. If this rule is not available, methods such as those in [53] can be employed to reconstruct the edges. In most cases, including this work, we assume that the method for edge formation in the graphs is predefined, which is defined on the graph signals. The proposed architecture of the GKAE and the operations of each of its components is depicted in Fig. 2, in which the GKAE's forward processing is given as:

$$\begin{aligned} G(t) &\xrightarrow{\text{POOL}(\mathcal{E}_{\theta}^{\text{GNN}}(\cdot))} \mathbf{g}(t) \xrightarrow{\mathcal{E}_{\theta}^{\text{KAE}}(\cdot)} \mathbf{h}(t) \\ &\xrightarrow{\mathbf{K}} \mathbf{h}(t+1) \xrightarrow{\mathcal{D}_{\phi}^{\text{KAE}}(\cdot)} \mathbf{g}(t+1) \xrightarrow{\mathcal{D}_{\phi}^{\text{GNN}}(\cdot)} \mathbf{x}(t+1). \end{aligned} \quad (27)$$

Training and Loss Function: When training the GKAE model, we approximate a linear representation of a non-linear dynamical system using the Koopman matrix. Since the input is a spatio-temporal graph, we first convert it into a compact lower-dimensional representation, which is the graph embedding. The non-linear dynamical system is modeled through the graph embedding, which captures information from multiple nodes in a lower-dimensional space, as illustrated in Fig. 2. Thus, during training, our objective is to develop graph embeddings

that retain minimal loss of information. These embeddings are subsequently transformed into the Koopman invariant subspace by the Koopman encoder to obtain a linear model up to L time steps. The Koopman decoder then maps the Koopman invariant space back into the graph embedding space. Following this, we train the graph decoder to reconstruct the graph signals, enabling recovery of the original spatio-temporal graph.

To minimize the information loss during the transformation from the spatio-temporal graph to the graph embedding and back, we incorporate a reconstruction loss, which is defined as

$$L_{\text{GNN}} = \sum_{t=0}^{T-1} \|\mathbf{x}(t) - \hat{\mathbf{x}}(t)\|^2, \quad (28)$$

Using (28) and (23), the aggregated loss function for the GKAE is given by:

$$\mathcal{L}_{\text{GKAE}}(\theta, \mathbf{K}, \phi, L) = L_{\text{GNN}} + \mathcal{L}_{\text{KAE}}(\theta, \phi, \mathbf{K}, L), \quad (29)$$

We use a sufficiently large value for L during training to ensure effective linear modeling of the graph embedding.

Linear Predictions using GKAE: The GKAE can be used to predict future time-varying graph signals in a linear subspace. This is quite straightforward due to its simple architecture.

We use P to denote the total number of steps, we aim to predict linearly. The value of P is determined based on the specific application requirements and is distinct from L , which represents the number of steps during the training to ensure linear modeling of the graph embeddings. The graph embedding for

the spatio-temporal graph at the first time step is given as

$$\mathbf{g}(0) = \mathcal{E}_{\theta_{\text{GNN}}}(\mathbf{x}(t), \mathcal{E}(t), \mathbf{W}(t)). \quad (30)$$

It is assumed that reconstructing the graph signals directly using the graph decoder should result in minimal reconstruction error, thereby ensuring that the learned embeddings effectively preserve the spatial characteristics of the original graph. The p -th time step prediction of the graph signals is now given using:

$$\hat{\mathbf{x}}(p) = \mathcal{D}_{\phi_{\text{GNN}}} \left(\mathcal{D}_{\phi_{\text{KAE}}} (\mathbf{K}^p (\mathcal{E}_{\theta_{\text{KAE}}} \mathbf{g}(0))) \right), \quad p = 1, 2, \dots, P. \quad (31)$$

IV. GKAE APPLICATION 2: TIME-VARYING GRAPH RECONSTRUCTION

In this section, we use the GKAE to address data loss and reconstruct missing node information in time-varying graph signals.

A. GKAE-Based Approach

The GKAE can be employed on limited historical data of τ time steps, which is fully observable

$$\mathbb{G} = [G(0), \dots, G(\tau - 1)], \quad (32)$$

where the objective is approximating a global linear model during training. With successful linear modeling, the future graph embeddings for unseen p -step predictions can be obtained as:

$$\mathbf{g}(\tau + p) = \mathcal{D}_{\phi_{\text{KAE}}} (\mathbf{K}^p (\mathcal{E}_{\theta_{\text{KAE}}} \mathbf{g}(\tau))), \quad p = 1, \dots, T - \tau - 1. \quad (33)$$

Beyond τ , the graph signals are sampled and only partially observable. Using GKAE, we can predict the graph embedding, as seen in (33), which is a representation of the fully observable graph signals in a latent space. The objective is to train a secondary autoencoder that maps the partially observable graph signals for each time step t to these predicted embeddings. The encoder of this secondary autoencoder takes the partially observable graph as input, while the decoder reconstructs the original graph signal. By aligning the output of the secondary autoencoder with the GKAE-predicted embeddings, the framework enables the reconstruction of the missing values in the original graph signals. This approach ensures the latent representation learned from the partially observable data aligns with the latent representation of the fully observable graph signals.

1) *Extending to Sampled Graph Input*: We propose a latent consistency autoencoder (LC-autoencoder) to process sampled spatio-temporal graphs and reconstruct the complete graph signals by aligning them with predicted graph embeddings, as illustrated in Fig. 3.

The LC-autoencoder consists of an encoder and decoder, which are denoted using $\mathcal{E}_{\theta_{\text{LC}}}$ and $\mathcal{D}_{\phi_{\text{LC}}}$, respectively, and are trained following the training of the GKAE. The encoder takes as input the sampled graph signals $\mathbf{Y}$ and map it to the predicted graph embedding. At time t , the masked input signals and corresponding adjacency matrix are defined as

$$\mathbf{y}'(t) = \mathbf{J}(:, t) \odot \mathbf{x}(t) \in \mathbb{R}^N \quad (34)$$

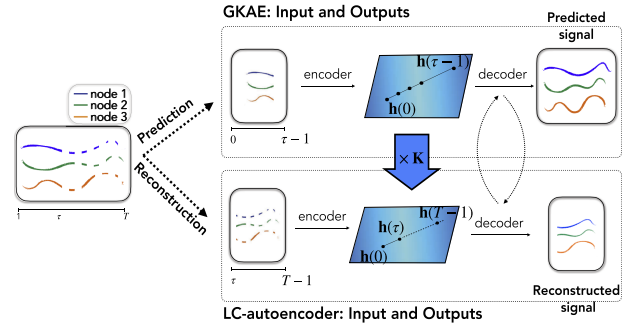


Fig. 3. Application of the GKAE for prediction and reconstruction using the LC-autoencoder, which processes sampled graph data while maintaining latent space consistency to reconstruct the original graph signals.

$$\mathbf{W}'(t) = \mathbf{W}(t) \odot (\mathbf{J}(:, t) \mathbf{J}(:, t)^\top) \in \mathbb{R}^{N \times N} \quad (35)$$

The encoder function of the LC-autoencoder takes the sampled input signals and the adjacency matrix and outputs a graph embedding, which is denoted using $\bar{\mathbf{g}}(t)$. The graph embedding is the input to the decoder of the LC-autoencoder, which reconstructs the complete graph signal using the decoder function. To achieve this, we enforce consistency in the encoder and decoder functions with the trained GKAE model. Since the input to the LC-autoencoder encoder differs from that of the GKAE model, consistency is ensured in the graph embedding rather than the parameters of the encoder. Once the same graph embedding is achieved with the sampled spatio-temporal graph, consistency between the parameters of the GKAE decoder and the LC-autoencoder decoder allows reconstruction of the complete graph signal. The flow of the LC-autoencoder is defined as:

$$\bar{\mathbf{g}}(t) = \mathcal{E}_{\theta_{\text{LC}}}(\mathbf{y}'(t), \mathbf{W}'(t)), \quad (36)$$

$$\bar{\mathbf{x}}(t) = \mathcal{D}_{\phi_{\text{LC}}}(\bar{\mathbf{g}}(t)), \quad (37)$$

2) *Training*: Our overall training approach combines inductive pre-training with a transductive reconstruction phase. More specifically, it consists of two steps:

- *GKAE Pre-training*: The GKAE is trained on the fully observed training set (i.e., time steps 0 to $\tau - 1$) to model system dynamics and predict future latent representations. The GKAE is trained using the loss function as in (29).
- *Transductive Reconstruction*: For this task, the LC-autoencoder is trained on the masked version of the test set. The pre-trained GKAE provides predicted target embeddings for this period, as seen in (33). The LC-autoencoder learns to map the incomplete input to these targets and reconstruct the corresponding graph signals. This process leverages information (the GKAE targets) derived from the unmasked test set itself, enabling accurate inpainting of missing values without generalizing to new, unseen masked datasets.

The parameters of these functions are optimized using the following loss functions:

$$L_{\text{latent}} = \sum_{t=0}^{T-1} \|\bar{\mathbf{g}}(t) - \mathbf{g}(t)\|^2, \quad (38)$$

$$L_{\text{cosine}} = \sum_{t=0}^{T-1} \left| 1 - \frac{\langle \bar{\mathbf{g}}(t), \mathbf{g}(t) \rangle}{\|\bar{\mathbf{g}}(t)\| \|\mathbf{g}(t)\|} \right|, \quad (39)$$

$$L_{\text{par}} = \sum_{t=\tau}^{T-1} \|\mathcal{D}_{\phi_{\text{LC}}}(\bar{\mathbf{g}}(t)) - \mathcal{D}_{\phi_{\text{GNN}}}(\mathbf{g}(t))\|^2, \quad (40)$$

where L_{latent} and L_{cosine} enforce *latent variable consistency*, while the term L_{par} ensures *decoder output consistency*. Therefore, even if some input data is missing due to masking or sampling limitations, as long as the same graph embedding can be generated, the sequences of unseen data can still be accurately reconstructed.

The final loss function for training the LC-autoencoder is given as:

$$\mathcal{L}_{\text{LC}} = \beta_1(L_{\text{latent}} + L_{\text{cosine}}) + \beta_2 L_{\text{par}}, \quad (41)$$

where β_1 and β_2 are hyperparameters controlling the relative importance of each loss component.

V. SIMULATION SETUP AND RESULTS

A. Baseline Methods

a) Reconstruction: State-of-the-art solutions for reconstructing graph signals include GSP techniques. Other than this, we also include *vanilla* GCN [33] and nearest neighbor interpolation (NNI) [54] as baselines. We consider the following baselines for comparison:

- *Temporal graph smoothness (TGS) [7]:* This approach leverages temporal smoothness constraints to reconstruct graph signals effectively. This method was trained using a gradient projection algorithm with backtracking line search, for up to 500 outer iterations or until convergence of the gradient norm. Each outer iteration adaptively selects a step size through at most 40 internal backtracking steps.
- *Temporal graph Sobolev smoothness (TGSS) [8]:* This approach extends TGS by incorporating Sobolev smoothness, capturing both temporal smoothness and higher-order variations for enhanced reconstruction. We use the same training settings as the TGS method.
- *D_X_L2/S_X_L2([24]):* This approach extends graph signal reconstruction over Type-2 and Type-3 time-varying graphs. Both the approaches D_X_L2 (dynamic graph structure) and S_X_L2 (static graph structure) are trained for 2000 iterations.

Other approaches include:

- *NNI [54].*
- *Vanilla GCN [33]:* We train an encoder-decoder architecture with 4 GCN layers. The method is trained for 500 epochs.

b) Prediction: For graph signal prediction tasks, we consider two sets of baselines. *Koopman-based* state-of-the-art methods primarily emphasize temporal dynamics but often overlook the spatial structure of graphs. We use the following methods as baselines:

- *KAE [12]:* Focuses on learning a Koopman-invariant subspace for linear modeling of temporal sequences. On average, the total number of parameters in this model is 3270.

- *Consistent KAE (cKAE) [37]:* Extends KAE by enforcing consistency in temporal evolution for better predictive performance by adding a backward prediction matrix. On average, the total number of parameters is slightly higher than the KAE.

These methods concatenate the input states of nodes to predict their future states; however, their disregard for the spatial structure of the graph significantly limits their capacity to model spatio-temporal dependencies effectively. While *GNN-based* approaches include:

- *Seq-2-Seq GNN with LSTM/GRU layers:* A graph-based encoder-decoder model that applies a GCN for spatial encoding and an LSTM or GRU for temporal modeling. Trained with approximately 18K–24 K parameters.
- *STGCN [42]:* A spatio-temporal model using two ST-Conv blocks [42], each with spatial graph convolutions and temporal convolutions. Trained with approximately 8 K parameters.
- *DCRNN [43]:* A recurrent architecture using one DCRNN cell in both the encoder and decoder, modeling spatial diffusion and temporal dynamics. Trained with approximately 8 K parameters.
- *CITRUS/CGCN [44]:* We use a feed-forward model with a linear input layer, four residual blocks, and an output layer. Each block uses spectral graph convolution with learnable time parameters and a small MLP. Trained with approximately 5 K parameters.

Unless specified otherwise, each baseline approach is trained for 500 iterations/epochs.

B. Datasets

We use several datasets for the empirical evaluation of our method against baseline approaches for both reconstruction and prediction tasks.

- *Type-1 time-varying graph:*
 - *SST NOAA [55]:* Global sea surface temperature (SST) measurements sampled weekly. The graph is static, constructed using the k -nearest neighbors (KNN) based on spatial proximity. For prediction, 300 weeks are used for training and 200 for testing. All experiments are conducted using 20 randomly selected nodes.
 - *METR-LA [43]:* Traffic speed data from loop detectors in Los Angeles, sampled every 5 minutes. Nodes represent sensors, and edges are defined by physical road connectivity. For prediction, 10 days are used for training and 4 days for testing.
 - *PEMS-BAY [43]:* Similar to METR-LA, with traffic speed readings from the Bay Area. The same prediction split is used.
 - *Beijing Air Quality [56]:* Meteorological and air quality data collected hourly from monitoring stations in Beijing. Nodes represent stations, with edges based on spatial proximity. We use atmospheric pressure (hPa) as the graph signal. For both prediction and reconstruction tasks, 300 hours are used for training and 200 for evaluation.

- *Type-3 time-varying graph: UAV [57]:* Simulated data of mobile UAVs, where the graph structure evolves over time as UAVs change neighbors based on mobility within a radius r . The graph signal is the average signal-to-noise ratio (SNR), sampled at 0.1-second intervals. For both prediction and reconstruction tasks, 30 seconds are used for training and 20 seconds for evaluation.

Note that since type-3 time-varying graph is a subset of type-2 time-varying graph, we do not consider a dataset of type-2 time-varying graph. We present results for both prediction and reconstruction tasks across the various datasets considered.

C. Performance Metrics

To evaluate the performance of the methods, we use the following metrics:

- 1) *Prediction Error:* The root mean-squared error (RMSE) and mean absolute error (MAE) for predictions are defined as:

$$\text{RMSE}_{\text{pred}} = \sqrt{\frac{1}{P} \sum_{p=1}^P \|\mathbf{x}(p) - \hat{\mathbf{x}}(p)\|^2}, \quad (42)$$

$$\text{MAE}_{\text{pred}} = \frac{1}{P} \sum_{p=1}^P \|\mathbf{x}(p) - \hat{\mathbf{x}}(p)\|_1, \quad (43)$$

where $\hat{\mathbf{x}}(p)$ is the predicted graph signal at time step p , and P is the number of predictions.

- 2) *Reconstruction Error:* The reconstruction loss, calculated over $T - \tau - 1$ steps on the masked input, is defined as:

$$\epsilon_{\text{recon}} = \frac{1}{T - \tau - 1} \sum_{t=\tau}^{T-1} \sum_{n: [\mathbf{J}]_{n,t}} \|\mathbf{x}_n(t) - \bar{\mathbf{x}}_n(t)\|^2, \quad (44)$$

where $\bar{\mathbf{x}}(t)$ is computed as described in (37).

D. Architecture of the GKAE

We use the PyTorch geometric library [58] for simulating our GKAE architecture.

a) *GKAE:* The graph encoder comprises two graph neural network layers: the first is a graph convolutional layer [59] that aggregates local neighborhood information using the graph structure and outputs a feature of dimension 8, and the second is a SAGEConv layer [60], which performs inductive node embedding by sampling and aggregating features from neighbors, also outputting features of dimension 8. Following the graph encoder, we apply a global mean pooling operator for a concise graph representation ($b = 8$). The Koopman autoencoder [37] operates entirely within the latent space and comprises 12 fully connected layers—6 in the encoder and 6 in the decoder, each with 16 hidden nodes. The latent dynamics are modeled using a learnable Koopman operator represented as a fully connected linear transformation of dimension $M \times M$ without an added bias term. The final reconstruction of the original graph signal is performed by the graph decoder which is composed of two fully connected layers applied to the Koopman-decoded latent space. We apply the Leaky ReLU activation function in both the graph

TABLE II
SIMULATION PARAMETERS

Model Architecture	
Size of Koopman matrix M	8
Size of graph embedding b	8
Epochs	200
Hyperparameters β_1, β_2	$1, 10^{-2}$
Linearity length L	50
Prediction length P	200
Masking ratio	50%
Length of observable data τ	300
Total length of data T	500
Optimizer	Adam
Learning rate	10^{-2}
Type-1 Datasets	
Number of nodes N	20.
Number of neighbors k	4
Type 3 Datasets	
Number of nodes N	20.
Communication radius of UAVs r	0.5
Transmit power	10 dBm
Path loss	2

encoder and decoder, while the Koopman autoencoder employs the hyperbolic tangent ($\tanh$) activation in all its fully connected layers. The total number of parameters for training the GKAE model is approximately 4K parameters.

b) *LC-autoencoder:* The LC-autoencoder is a secondary model specifically trained to handle the reconstruction of incomplete data. We use 4 layers, which is equally divided between the encoder and the decoder. The encoder processes the masked input with SAGECONV layers [60], which is followed by a global pooling operator to generate the same graph embedding of dimension b . We use the Leaky ReLU activation function in this architecture as well. The LC-autoencoder consists of approximately 3K parameters.

Unless specified otherwise, the parameters for defining our simulation environment is declared in Table II.

E. Results

1) *Prediction via Graph Koopman Autoencoder:* In this section, we present the results for predicting P steps over various datasets, which include synthetic dataset (UAV-SNR), traffic prediction dataset (METR-LA & PEMS-BAY) and environmental datasets (SST NOAA & Beijing Air Quality).

Best Koopman dimension: Firstly, in Fig. 4, we compare the loss during training over varying dimension of the $\mathbf{K}$ matrix. The dimension of the $\mathbf{K}$ matrix is critical as it determines the dimensionality of the Koopman invariant subspace, which directly influences the model's ability to capture and predict the underlying dynamics. A matrix that is too small may fail to capture the necessary complexity of the system, leading to inaccurate predictions, while an excessively large matrix may introduce redundancy and increase computational overhead. A high dimensional $\mathbf{K}$ matrix may also cause instability during training, particularly with long trajectory lengths due to recursive multiplication, as seen in the case with $M = 32$.

TABLE III
PREDICTION PERFORMANCE ACROSS DIFFERENT MODELS AND DATASETS. EACH CELL REPORTS THE PERFORMANCE METRIC AVERAGED OVER 10 EVALUATION RUNS WHICH IS EVALUATED ON THE TEST SET.

Model	SNR (2s)		SST (6 months)		METR-LA (60m)		PEMS-BAY (60m)		Beijing Pressure (24h)	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
KAE [12]	0.79	0.53	1.26	0.86	8.96	6.56	5.57	3.59	4.28	3.65
cKAE [37]	0.30	0.20	2.56	1.84	8.98	6.50	5.53	3.52	4.42	3.78
GCN-GRU	0.23	0.19	2.71	1.91	9.30	7.05	5.37	3.51	4.54	4.03
GCN-LSTM	0.24	0.18	1.52	1.20	9.30	6.80	5.44	3.40	4.01	3.60
STGCN [42]	0.51	0.37	3.34	2.37	8.71	5.94	5.08	3.13	4.89	4.08
cGCN [44]	0.43	0.28	4.41	3.80	11.8	9.62	8.25	6.40	6.35	5.79
DCRNN [43]	0.30	0.23	1.95	1.36	8.42	5.81	5.63	3.60	5.82	5.04
GKAE (Ours)	0.11	0.06	1.12	0.78	8.31	5.78	4.12	2.88	3.79	3.32

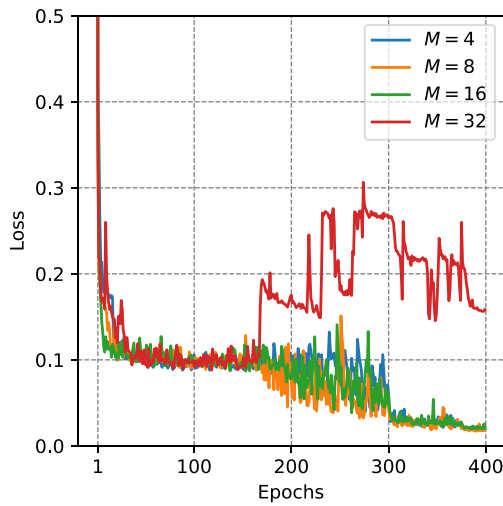


Fig. 4. Loss convergence over varying dimensions for different Koopman matrix dimensions M .

In Table III, we use a Koopman dimension of $M = 8$ for training our models and compare them against the aforementioned baselines for long-horizon predictions.

On the UAV-SNR dataset, we forecast 2 seconds into the future, where our model significantly outperforms all baselines. Specifically, it achieves a 52.2% reduction in RMSE and a 66.7% reduction in MAE compared to GCN-LSTM.

On the SST dataset, our method improves long-range (6-month) predictions, achieving an 11.1% reduction in RMSE and a 9.3% reduction in MAE. While GCN-LSTM also shows competitive performance, our model outperforms it by 26.3% in RMSE and 35% in MAE.

For the more challenging traffic forecasting tasks, GKAE also demonstrates notable improvements. On the PEMS-BAY dataset, it reduces RMSE by 18.9% and MAE by 8.0% compared to the next-best model, STGCN. Although the performance margin is narrower on the METR-LA dataset, our model still surpasses all baselines.

Finally, on the pressure dataset, our approach outperforms GCN-LSTM by 5.5% in RMSE and 7.8% in MAE. We also evaluate the RMSE and MAE over a varying number of nodes

in the data for the three best-performing models on each type of dataset in Fig. 5. Firstly, as depicted in Fig. 5(a) and (d), our model outperforms the DCRNN by 1.31-2.94% in RMSE and 0.52-3.01% in MAE, on average. The performance improvement is more pronounced in the SST and UAV-SNR datasets, where our model outperforms the next-best models by a significant margin. Specifically, on the SST dataset, it improves upon the GCN-LSTM baseline by 15.7-19.0% in RMSE and 18.8-23.5% in MAE, represented in Fig. 5(b) and (e), respectively. The advantage is even more substantial on the UAV-SNR dataset. It is shown in Fig. 5(c) and (f) that our model outperforms the strongest baselines by 47.6-54.1% in RMSE and 61.5-68.1% in MAE.

In Fig. 6, we compare the predicted trajectories against the ground truth over the test set for the specified horizon lengths, which shows that the GKAE can perform well for predicting on different types of datasets.

2) *Reconstruction Using LC-Autoencoder*: For the reconstruction task, our objective is to accurately inpaint missing node information within time-varying graph signals. We employ the LC-autoencoder for this purpose, which reconstructs masked graph signals by aligning their latent space and decoder output with the representations provided by the pre-trained GKAE model. The detailed training and evaluation methodology for the LC-autoencoder, including its transductive learning framework and specific data utilization, is presented in Section IV. Reconstruction performance is evaluated on its ability to reconstruct the masked graph signals in the test set. The following section presents a detailed comparison of reconstruction errors across multiple methods and datasets.

a) *Generalization*: In Fig. 7, we evaluate the reconstruction capability of the LC-autoencoder. For this specific analysis of transductive inpainting, the figure presents results on a distinct subset of the data that serves to demonstrate generalization to unmasked or partially observed samples, labeled within the figure as the ‘validation set.’ This allows for an in-depth examination of the model’s performance on both masked and unmasked nodes, as well as a comparison between its performance on the training data and this particular subset.

The reconstruction error is compared for masked and unmasked nodes across different masking levels. While the error

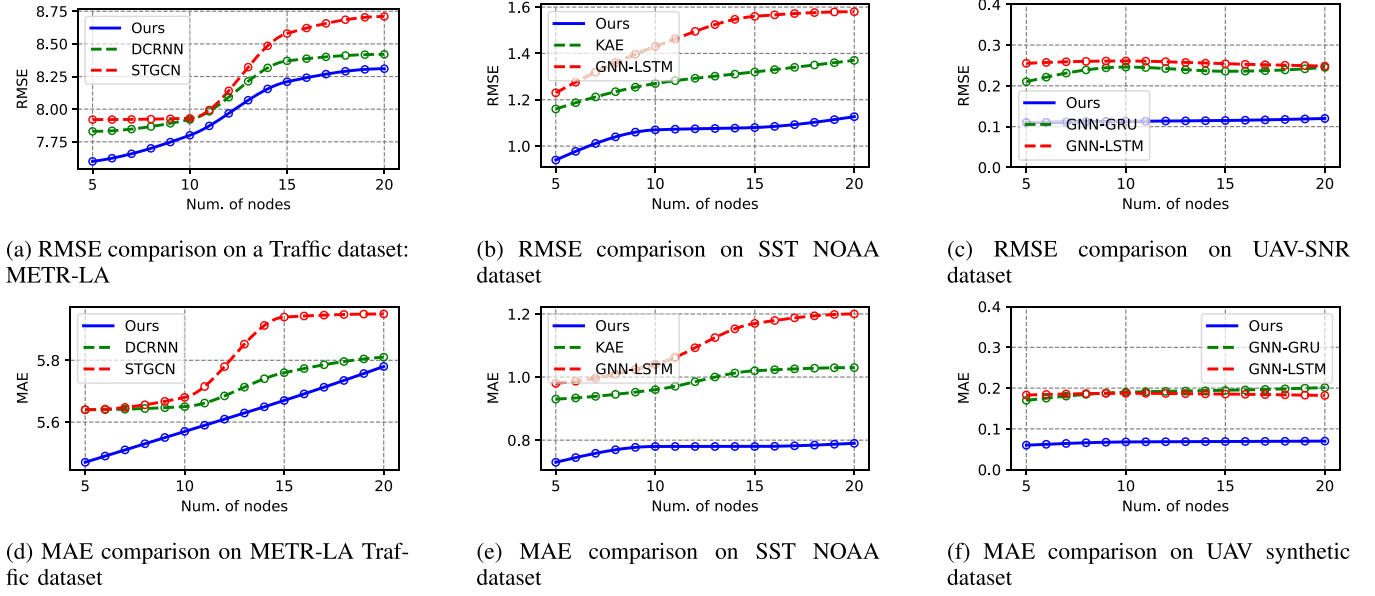


Fig. 5. Performance comparisons over three dataset types over varying number of nodes N , compared for (top row) RMSE, (bottom row) MAE. The best three models have been compared.

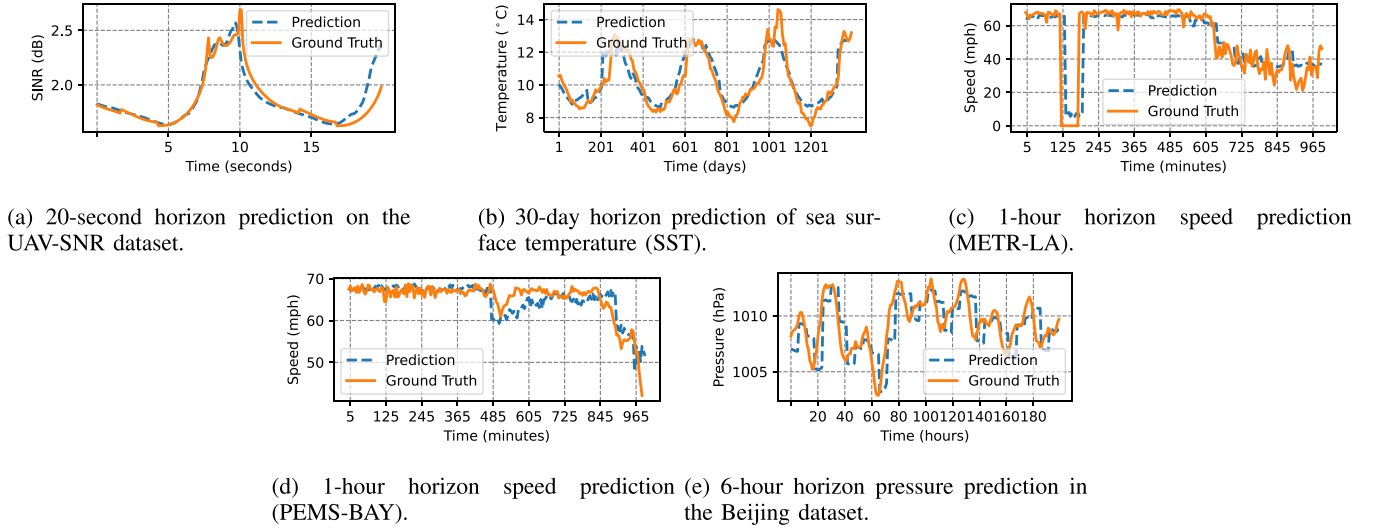


Fig. 6. 200-step sequential prediction comparison across datasets with varying forecasting horizons, showing ground truth and predicted trajectories.

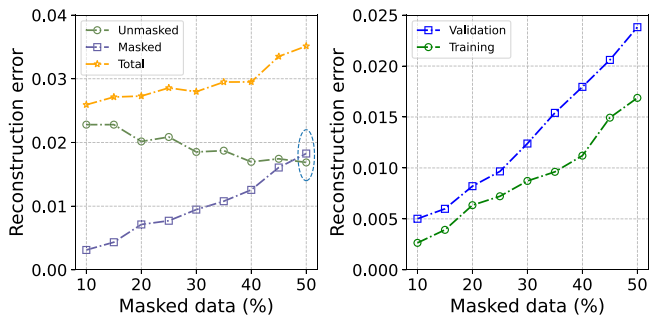


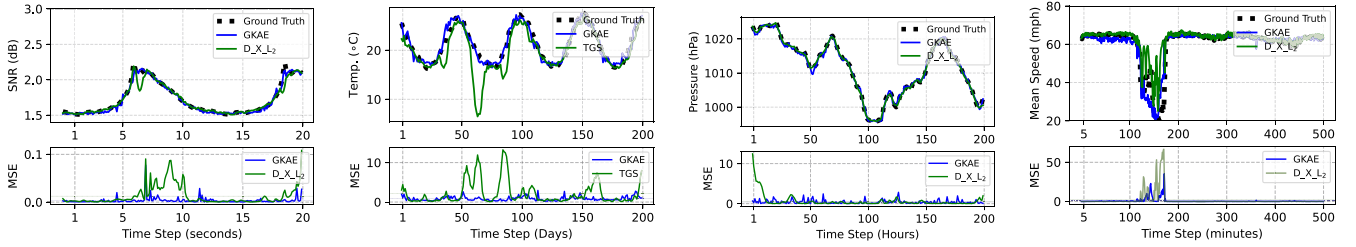
Fig. 7. Evaluating the generalization ability of the GKAE for reconstruction by comparing the reconstruction of masked/unmasked nodes (left) and training/validation data (right).

increases with higher proportions of masked data, it remains comparable when masking 50% of the nodes, demonstrating the LC-autoencoder's ability to effectively reconstruct both visible and masked inputs. We also compare the reconstruction performance on the training and validation sets, noting that the validation reconstruction error is slightly higher than the training reconstruction error.

b) Comparison with Baselines on Varying Datasets: For the reconstruction task, we present results on multiple datasets across different masking rates, summarized in Table IV. It is worth noting that the ϵ_{recon} has been reported as the performance metric. It can be noted that our approach outperforms on almost every setting and dataset. In general, it can be seen that the TGSS [8] and D_X_L2 offer comparable performance in most of

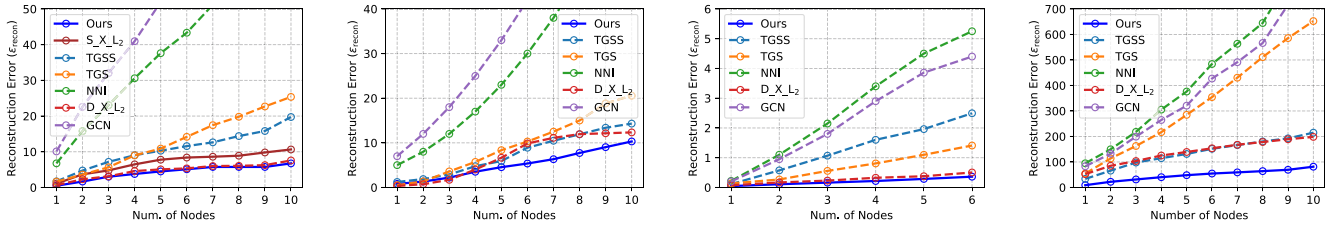
TABLE IV
RECONSTRUCTION PERFORMANCE (ϵ_{recon}) ACROSS DIFFERENT MASKING RATES AND DATASETS

Model	SNR (UAV)					SST					Beijing Pressure					PEMS-BAY				
	10%	20%	30%	40%	50%	10%	20%	30%	40%	50%	10%	20%	30%	40%	50%	10%	20%	30%	40%	50%
TGS [7]	0.004	0.006	0.017	0.048	0.127	0.052	0.477	0.633	1.047	2.057	0.137	0.268	0.557	0.808	1.098	54.62	162.63	284.75	430.46	585.78
TGSS [8]	0.003	0.010	0.023	0.046	0.099	0.123	0.572	0.773	1.191	1.431	0.112	0.575	1.068	1.601	1.962	65.96	114.82	151.61	180.46	214.85
D_X_L2 [24]	0.007	0.009	0.019	0.033	0.038	0.073	0.402	0.986	1.195	1.234	0.074	0.169	0.231	0.326	0.377	85.04	125.24	153.60	178.22	198.30
GCN [33]	0.088	0.208	0.312	0.456	0.590	0.654	1.380	2.180	2.945	3.432	0.703	1.557	2.635	3.814	4.479	105.23	190.45	325.78	488.64	699.91
NNI [54]	0.080	0.174	0.236	0.326	0.406	0.944	2.012	3.341	4.167	4.985	1.027	2.185	3.709	5.043	5.962	123.61	207.78	341.09	512.83	742.50
S_X_L2 [24]	0.008	0.011	0.022	0.036	0.053	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
GKAE (Ours)	0.002	0.003	0.012	0.030	0.034	0.132	0.353	0.537	0.891	1.031	0.047	0.108	0.162	0.218	0.287	22.18	40.31	54.65	64.04	81.44



(a) Reconstruction over the test set and the MSE over time on the UAV-SNR dataset for GKAE vs D_X_L2. (b) Reconstruction over the test set and the MSE over time on the SST dataset for GKAE vs TGS over 200 time steps. (c) Reconstruction over the test set and the MSE over time on the pressure dataset for GKAE vs D_X_L2. (d) Reconstruction over the test set and the MSE over time on the PEMS-BAY dataset for GKAE vs D_X_L2.

Fig. 8. Reconstruction comparison with MSE over time for the two best performing methods over varying datasets.



(a) Reconstruction error for the various methods over varying number of nodes on the UAV-SNR dataset. (b) Reconstruction error for the various methods over varying number of nodes on the SST dataset. (c) Reconstruction error for the various methods over varying number of nodes on the Beijing pressure dataset. (d) Reconstruction error for the various methods over varying number of nodes on the PEMS-BAY traffic dataset.

Fig. 9. Reconstruction error over varying datasets for different methods.

the scenarios. On average, our approach outperforms the TGSS by 57.4% , while on the other hand, it surpasses the D_X_L2 model by an average of 43.5% across all applicable settings and datasets. The highest improvement was seen with a 64.9% better performance than the TGSS model on the PEMS-BAY dataset at a 20% masking rate, while the lowest was a 9.1% improvement over the D_X_L2 model on the SNR dataset at a 40% masking rate. It is worth noting that the GCN and NNI do not offer good performance, while the TGS offers comparable performances when the masking rate is low.

In Fig. 8, we present reconstruction plots over the test set for the two best-performing methods on each dataset. For the UAV-SNR dataset, our approach is compared against D_X_L2, which is designed to handle time-varying graph structures. Similarly, D_X_L2 is used as a baseline for the Beijing pressure and PEMS-BAY traffic datasets, while TGS is used for comparison on the SST dataset.

On average, our model outperforms the next-best method by 12.4-56.4% on the UAV-SNR dataset, 28.0-37.8% on the SST dataset, 3.5-27.8% on the Beijing pressure dataset, and 58.9-82.7% on the PEMS-BAY dataset.

c) Completely Masked Nodes: The previous results highlighted scenarios where masking is time-variant and sufficient unmasked data is available for each node. However, there may be instances during training where no information is available for certain nodes. Baseline methods, which rely on temporal smoothness to reconstruct missing values, require adequate unmasked input for effective training. In contrast, when a node is completely masked across all time instances, these methods struggle to accurately reconstruct the graph signals. Our approach, however, leverages spatial learning through the GNN to accurately predict the graph embedding. This capability allows our method to reconstruct the graph embedding even when a node remains entirely masked during training.

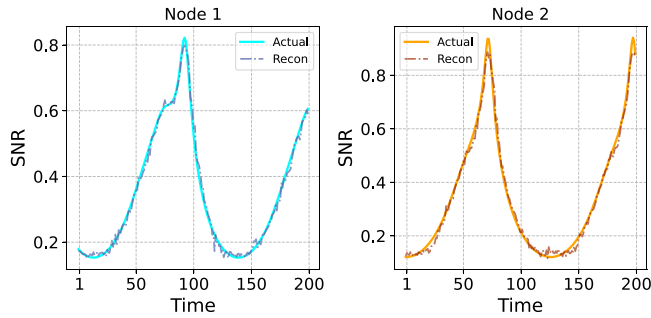


Fig. 10. Reconstruction of graph signals when the graph signals for these nodes are completely masked during training.

In Fig. 10, we illustrate the reconstructed graph signals for two nodes that were completely masked during training. Despite the absence of any training data for these nodes, our method demonstrates the ability to reconstruct their signals with low error. This is achieved by leveraging the matching of latent variables through the graph embeddings, which capture the spatial relationships and dynamics of the graph. This result highlights the robustness of our approach in handling scenarios where no temporal information is available for certain nodes, effectively utilizing the graph structure and spatial learning to ensure accurate reconstructions.

VI. CONCLUSION

In this work, we proposed a novel approach by leveraging GNNs and Koopman theory to address reconstruction and prediction tasks on time-varying graphs with dynamic structures. Unlike previous methods, which are typically restricted to time-varying graphs with static structures (e.g., where only node features vary over time while graph properties remain fixed, failing to capture dynamic relationships between nodes), our approach captured the evolving dynamics of node relationships through a flexible latent-space framework grounded in Koopman theory. This framework introduced a graph embedding technique applicable to any type of time-varying graph.

We evaluated the performance of our method against well-established baselines and demonstrated an average improvement of 57.4% in reconstruction tasks and 18.1% in long-term prediction tasks. Future work will focus on enhancing consistency and further improving long-term prediction accuracy.

REFERENCES

- [1] E. Estrada, *The Structure of Complex Networks: Theory and Applications*. London, U.K.: Oxford Univ. Press, Dec. 2016.
- [2] N. Henry and J.-D. Fekete, "MatrixExplorer: A dual-representation system to explore social networks," *IEEE Trans. Vis. Comput. Graph.*, vol. 12, no. 5, pp. 677–684, Sep./Oct. 2006.
- [3] M. M. Li, K. Huang, and M. Zitnik, "Graph representation learning in biomedicine and healthcare," *Nat. Biomed. Eng.*, vol. 6, pp. 1353–1369, Dec. 2022.
- [4] X. Huang, Y. Zhao, C. Ma, J. Yang, X. Ye, and C. Zhang, "TrajGraph: A graph-based visual analytics approach to studying urban network centralities using taxi trajectory data," *IEEE Trans. Vis. Comput. Graph.*, vol. 22, no. 1, pp. 160–169, Jan. 2016.
- [5] J. Suárez-Varela et al., "Graph neural networks for communication networks: Context, use cases and opportunities," *IEEE Netw.*, vol. 37, no. 3, pp. 146–153, May/Jun. 2023.
- [6] V. Nicosia, J. Tang, M. Musolesi, G. Russo, C. Mascolo, and V. Latora, "Components in time-varying graphs," *Chaos*, vol. 22, Jun. 2012, Art. no. 23101.
- [7] K. Qiu, X. Mao, X. Shen, X. Wang, T. Li, and Y. Gu, "Time-varying graph signal reconstruction," *IEEE J. Sel. Topics Signal Process.*, vol. 11, no. 6, pp. 870–883, Sep. 2017.
- [8] J. H. Giraldo, A. Mahmood, B. Garcia-Garcia, D. Thanou, and T. Bouwmans, "Reconstruction of time-varying graph signals via Sobolev smoothness," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 8, pp. 201–214, 2022.
- [9] J. A. Castro-Correa, J. H. Giraldo, M. Badié, and F. D. Malliaros, "Gegenbauer graph neural networks for time-varying signal reconstruction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11734–11745, Sep. 2024.
- [10] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Nat. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.
- [11] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci. USA*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [12] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature Commun.*, vol. 9, no. 1, 2018, Art. no. 4950.
- [13] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, pp. 309–325, Aug. 2005.
- [14] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, "Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control," *PLoS One*, vol. 11, pp. 1–19, Feb. 2016.
- [15] I. Mezić, "Koopman operator, geometry, and learning of dynamical systems," *Notices Amer. Math. Soc.*, vol. 68, pp. 1087–1105, Aug. 2021.
- [16] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.
- [17] N. Takeishi, Y. Kawahara, and T. Yairi, "Learning Koopman invariant subspaces for dynamic mode decomposition," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Curran Associates Inc., 2017, pp. 1130–1140.
- [18] S. Krishnan, J. Park, G. Sherman, B. Campbell, and J. Choi, "Predictive covert communication against multi-UAV surveillance using graph Koopman autoencoder," *IEEE Trans. Veh. Technol.*, vol. 74, no. 6, pp. 9165–9179, Jun. 2025.
- [19] S. Krishnan, J. Park, and J. Choi, "Graph signal reconstruction via Koopman autoencoder," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2025, pp. 1–5.
- [20] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, May 2013.
- [21] A. Sandryhaila and J. M. F. Moura, "Discrete signal processing on graphs," *IEEE Trans. Signal Process.*, vol. 61, no. 7, pp. 1644–1656, Apr. 2013.
- [22] A. Ortega, P. Frossard, J. Kováčević, J. M. Moura, and P. Vandergheynst, "Graph signal processing: Overview, challenges, and applications," *Proc. IEEE*, vol. 106, no. 5, pp. 808–828, May 2018.
- [23] G. Leus, A. G. Marques, J. M. Moura, A. Ortega, and D. I. Shuman, "Graph signal processing: History, development, impact, and outlook," *IEEE Signal Process. Mag.*, vol. 40, no. 4, pp. 49–60, Jun. 2023.
- [24] E. Yamagata, K. Naganuma, and S. Ono, "Robust time-varying graph signal recovery for dynamic physical sensor network data," *Trans. Signal Inf. Process. Netw.*, vol. 11, pp. 59–70, 2025, doi: [10.1109/TSIPN.2025.3525978](https://doi.org/10.1109/TSIPN.2025.3525978)
- [25] P. Goyal and E. Ferrara, "Graph embedding techniques, applications, and performance: A survey," *Knowl.-Based Syst.*, vol. 151, pp. 78–94, 2018.
- [26] M. Xu, "Understanding graph embedding methods and their applications," *SIAM Rev.*, vol. 63, no. 4, pp. 825–853, 2021.
- [27] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural Comput.*, vol. 15, no. 6, pp. 1373–1396, 2003.
- [28] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [29] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.
- [30] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.

- [31] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric transitivity preserving graph embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1105–1114.
- [32] H. Dai, B. Dai, and L. Song, "Discriminative embeddings of latent variable models for structured data," in *Proc. 33rd Int. Conf. Int. Conf. Mach. Learn.*, 2016, pp. 2702–2711.
- [33] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.
- [34] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2018.
- [35] S. Chen and Y. C. Eldar, "Time-varying graph signal inpainting via unrolling networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2021, pp. 8092–8097.
- [36] J. A. Castro-Correa, J. H. Giraldo, A. Mondal, M. Badié, T. Bouwmans, and F. D. Malliaros, "Time-varying signals recovery via graph neural networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [37] O. Azencot, N. B. Erichson, V. Lin, and M. Mahoney, "Forecasting sequential data using consistent koopman autoencoders," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 475–485.
- [38] J. Choi, S. Krishnan, and J. Park, "Koopman autoencoder via singular value decomposition for data-driven long-term prediction," in *Proc. IEEE 34th Int. Workshop Mach. Learn. Signal Process. (MLSP)*, 2024, pp. 1–6.
- [39] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2013, pp. 1310–1318.
- [40] J. Schmidhuber et al., "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [41] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (GRU) neural networks," in *Proc. IEEE 60th Int. Midwest Symp. Circuits Syst. (MWSCAS)*, 2017, pp. 1597–1600.
- [42] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640, doi: [10.24963/ijcai.2018/505](https://doi.org/10.24963/ijcai.2018/505).
- [43] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2018.
- [44] A. Einizade, F. Malliaros, and J. H. Giraldo, "Continuous product graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 90226–90252.
- [45] F. R. K. Chung, *Spectral Graph Theory*. Providence, RI, USA: Amer. Math. Soc., 1997.
- [46] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, 2011.
- [47] D. Spielman, "Spectral graph theory," in *Combinatorial Scientific Computing*, ch. 16, Philadelphia, PA, USA: Chapman & Hall/CRC, Oct. 2019.
- [48] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Dec. 2024, doi: [10.1109/TPAMI.2024.3443141](https://doi.org/10.1109/TPAMI.2024.3443141).
- [49] M. Budisić, R. Mohr, and I. Mezić, "Applied kism," *Chaos*, vol. 22, Dec. 2012, Art. no. 047510.
- [50] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, "On dynamic mode decomposition: Theory and applications," *J. Comput. Dyn.*, vol. 1, no. 2, pp. 391–421, 2014.
- [51] J. N. Kutz, S. L. Brunton, B. W. Brunton, and J. L. Proctor, *Dynamic Mode Decomposition*. Philadelphia, PA, USA: Soc. Ind. Appl. Math., 2016.
- [52] Y. Shi, Y. Dong, Q. Tan, J. Li, and N. Liu, "GiGaMAE: Generalizable graph masked autoencoder via collaborative latent space reconstruction," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 2259–2269.
- [53] T. N. Kipf and M. Welling, "Variational graph auto-encoders," in *NeurIPS Workshop Bayesian Deep Learn.*, 2016.
- [54] R. Sibson, "A brief description of natural neighbour interpolation," in *Proc. Interpreting Multivariate Data*, V. Barnett, Ed. Chichester U.K.: Wiley, 1981, pp. 21–36.
- [55] NOAA Physical Sciences Laboratory, "NOAA OI SST V2 high resolution dataset," 2024. [Online]. Available: <https://psl.noaa.gov/>
- [56] S. Chen, "Beijing multi-site air quality," *UCI Machine Learning Repository*, 2017. [Online]. Available: <https://doi.org/10.24432/C5RK5G>
- [57] X. Shao, H. Liu, W. Zhang, J. Zhao, and Q. Zhang, "Path driven formation-containment control of multiple UAVs: A path-following framework," *Aerosp. Sci. Technol.*, vol. 135, 2023, Art. no. 108168.
- [58] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *Proc. ICLR Workshop Represent. Learn. Graphs Manifolds*, 2019.
- [59] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.
- [60] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.